MILP-aided Trail Selection for Efficient Full Key Recovery in SPN ciphers

No author given*
no.email.given

Abstract

In this paper, we present an efficient MILP-aided key recovery framework that enables the systematic selection of differential trails with minimized subkey enumeration cost. Our approach formulates trail selection as an optimization problem, allowing the cryptanalyst to balance statistical strength and computational complexity in a principled manner. The proposed technique is generic and applicable to a wide range of symmetric-key primitives, particularly SPN-based block ciphers. As a case study, we demonstrate a full key recovery attack on the FUTURE block cipher using our optimized multi-trail strategy. Additionally, we report a reduced number of Work Units (WUs) in the MILP-based trail search compared to previous approaches, highlighting the practical efficiency of our method.

Keywords

FUTURE cipher, Lightweight block cipher, SPN, Differential cryptanalysis, Linear cryptanalysis, Nibble-based abstraction, Truncated model

ACM Reference Format:
No author given. 2026. MILP-aided Trail Selection for Efficient Full Key Recovery in SPN ciphers. In . ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Lightweight block ciphers are a fundamental building block in resource-constrained environments such as IoT devices, embedded systems, and RFID applications. Among the various design paradigms, Substitution–Permutation Networks (SPNs) built from 4-bit S-boxes remain particularly attractive due to their compact hardware footprint, efficient software realization, and well-understood diffusion properties. Ensuring the resistance of such designs against differential and linear cryptanalysis is therefore essential.

A central task in modern cryptanalysis is the construction of multi-round propagation trails. A *trail* describes the evolution of input–output differences (or linear masks) across consecutive rounds of a cipher and reveals structured statistical dependencies exploitable in key-recovery attacks. Over the years, several refinements of differential cryptanalysis have been proposed to improve its practicality. In particular, truncated differential cryptanalysis, introduced by Knudsen [10], considers partially specified differences that represent sets of classical differentials. Since its introduction, truncated techniques have been successfully applied and refined in numerous works [2, 5, 13]. More recently, Ahmadian *et al.* [1] provided new theoretical insights into truncated differentials for SPN ciphers with MDS diffusion layers and demonstrated their effectiveness on members of the QARMA family [3].

With the advent of automated cryptanalysis, Mixed-Integer Linear Programming (MILP) has become a powerful tool for constructing optimal differential and linear trails [11]. MILP-based frameworks systematically search for trails minimizing weight (or maximizing probability) and have been successfully applied to a wide range of block ciphers [7]. However, most existing approaches operate at the bit level and treat each state bit independently. While this modeling choice ensures accuracy, it scales poorly for full-round analyses of SPNs whose internal components naturally operate on 4-bit nibbles. For such ciphers, bit-level formulations introduce redundant constraints and unnecessary computational overhead.

This limitation is particularly relevant for lightweight 4-bit SPN designs. In these ciphers, the S-box layer, diffusion layer, and permutation layer are all defined on nibble boundaries. Modeling propagation at the bit level fails to exploit this structural granularity and complicates full-round trail construction. As a consequence, many analyses remain restricted to reduced-round variants.

The lightweight block cipher FUTURE [8] is a recent example of a 4-bit S-box based SPN. Its S-box layer, 16-bit diffusion layer, and ShiftRow permutation strictly preserve nibble alignment. Despite its structured design, no full-round differential or linear key-recovery attack has been reported. Existing evaluations are limited to partial-round analysis; even the designers analyzed only up to half of the rounds and extrapolated one additional round. The absence of full-round results highlights the difficulty of scaling conventional bit-level approaches.

More importantly, although numerous works employ MILP to construct optimal trails with respect to differential probability or linear weight, they do not consider an equally crucial aspect of practical cryptanalysis: the subkey enumeration cost during full key recovery. In existing literature, MILP is primarily used to identify the "best" distinguishers. The subsequent key-recovery phase is treated separately, without integrating the complexity of subkey guessing into the optimization process. To the best of our knowledge, no prior work explicitly incorporates subkey enumeration cost into the MILP formulation for full key recovery.

Our Contributions. In this work, we bridge the gap between trail optimization and efficient full-key recovery. Our main contributions are summarized as follows:

*No author given

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- We propose a nibble-level MILP framework tailored to 4-bit SPN block ciphers, reducing modeling overhead and improving scalability.
- We introduce an MILP-aided key-recovery methodology that selects differential trails while explicitly minimizing the overall subkey enumeration cost.
- We apply the proposed framework to the FUTURE block cipher and provide the first full-round key-recovery analysis.
- We achieve a reduction in overall Work Units (WUs) compared to previous analyses, demonstrating the practical advantage of integrating subkey-cost minimization into automated cryptanalysis.

Our approach shows that optimizing trail probability alone is insufficient for efficient full key recovery. By jointly considering propagation quality and subkey enumeration complexity, we obtain more practical attacks and provide a refined perspective on MILP-aided cryptanalysis of lightweight SPNs.

1.1 Challenges in Previous Approaches and Structural Opportunity in FUTURE

Prior cryptanalytic evaluations of FUTURE predominantly relied on bit-level truncated differential models. While such formulations are general and precise, they are structurally misaligned with the architectural principles of the cipher.

FUTURE is a 4-bit SPN in which all internal components operate strictly at nibble granularity:

- the substitution layer consists of 4-bit S-boxes,
- the linear diffusion layer processes columns of nibbles,
- the ShiftRow permutation preserves nibble boundaries.

Despite this clearly nibble-aligned structure, conventional modeling strategies treat each bit of the internal state independently. This abstraction mismatch introduces unnecessary complexity and results in:

- a significant increase in modelling dimensionality,
- redundant exploration of bit configurations that are equivalent at the nibble level,
- rapid growth in linear constraints required to encode XOR and diffusion relations,
- limited scalability of MILP-based trail construction beyond reduced rounds.

Consequently, prior analyses were confined to reduced-round settings. Full-round security arguments relied on partial propagation studies supplemented by heuristic extrapolations. The primary limitation was therefore not the attack paradigm itself, but the inefficiency of the underlying modeling framework.

Structural Opportunity. Interestingly, the same design regularity that makes FUTURE compact and hardware-friendly also creates a modeling opportunity. Because all nonlinear and linear transformations preserve nibble boundaries, difference propagation can be abstracted at the nibble level without loss of structural accuracy.

This structural property significantly reduces combinatorial complexity:

- the state space shrinks by a factor of four compared to bit-level abstraction,
- equivalence classes of bitwise patterns collapse into single nibble states,
- diffusion constraints become more compact and solver-friendly.

In this sense, FUTURE's strict nibble alignment weakens resistance against propagation-based automation: differential trail construction becomes substantially more efficient when the abstraction level matches the cipher's natural granularity.

Why FUTURE?. We deliberately select FUTURE as a case study because it represents an ideal testbed for demonstrating the power of nibble-level MILP modeling. Its structure exposes the inefficiency of bitwise formulations while allowing a clean abstraction that preserves exact propagation behavior. This makes full-round trail construction not only feasible but computationally efficient under an appropriately aligned model.

The present work leverages this structural opportunity by introducing a nibble-based truncated abstraction tailored to 4-bit SPNs. By aligning the modeling level with the cipher's intrinsic design, we significantly reduce solver complexity and enable exact full-round propagation analysis—something that remained impractical under previous bit-level approaches.

1.2 On the Importance of Full-Round Differential Trail Construction

For modern lightweight block ciphers, constructing differential trails across the full number of rounds remains a challenging objective. In many cases, including FUTURE, existing analyses are limited to reduced-round distinguishers. The original designers evaluated approximately half of the total rounds and extrapolated the remaining security margin heuristically. While such reduced-round studies provide useful insight, they do not fully characterize the cipher's round-by-round propagation behavior.

From a key-recovery perspective, reduced-round trails are insufficient. Full-round propagation analysis is necessary to accurately determine:

- the minimum number of active S-boxes across all rounds,
- the cumulative diffusion structure,
- the precise locations where subkey material influences the state.

These factors directly impact the complexity of subkey enumeration during a key-recovery attack. In particular, the distribution of active S-boxes in the outer rounds determines how many key bits must be guessed and how effectively wrong keys can be filtered.

Moreover, without a full-round study, it is difficult to exclude the possibility of structural effects such as differential clustering or non-uniform activity propagation in the final rounds. Such effects may either strengthen or weaken practical attacks, but they cannot be assessed through partial-round analysis alone.

The main obstacle to full-round trail construction has been modeling scalability. Bit-level MILP formulations grow rapidly in complexity and often become impractical beyond reduced-round settings. By introducing a nibble-aligned abstraction tailored to the structure of FUTURE, we overcome this limitation and enable efficient full-round differential trail construction.

This full-round analysis forms the foundation of our MILP-aided key-recovery framework. Rather than relying on heuristic security

margins, we derive exact propagation characteristics across all rounds and integrate them into a subkey-cost-aware optimization strategy.

Organization. Section 2.1 describes the structure of FUTURE and Section 2.2 reviews truncated cryptanalysis. Our nibble-based framework is introduced in Section 3. The full-round differential trails obtained using our method are presented in Section 4.3. Section 7 discusses the work units required during trail construction, and Section 8 concludes the paper.

2 Background

In this section, we present the foundational concepts required to follow the developments in the paper.

2.1 Preliminaries

In this section, we recall the structure of the lightweight block cipher FUTURE [8]. The description is kept minimal and limited to the components relevant to our nibble-based truncated analysis.

2.1.1 Notation. We denote by $\{0,1\}^n$ the set of n-bit binary strings. A nibble refers to a 4-bit value. The 64-bit internal state of FUTURE is represented as a 4×4 matrix of nibbles

$$S = \begin{pmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{pmatrix}, \qquad s_i \in \{0,1\}^4.$$

This representation makes the nibble boundaries explicit and reflects how each round transformation operates.

2.1.2 Structural details. FUTURE is a 10-round 64-bit SPN cipher with a 128-bit master key. Each round consists of four steps.

$$\text{SubCell} \rightarrow \text{MixColumn} \rightarrow \text{ShiftRow} \rightarrow \text{AddRoundKey}.$$

The final round omits the MixColumn layer. The master key K is split into two 64-bit halves K_0 and K_1, where K_0 is used as the whitening key and subsequent round keys are generated by 5-bit cyclic rotations of K_0 and K_1.

2.1.3 SubCell. The non-linear layer applies a 4-bit S-box S independently to each nibble of the state:

$$s_i \leftarrow S(s_i), \qquad 0 \le i < 16.$$

The S-box is defined as the composition of four lightweight S-boxes and has algebraic degree 3. Since this transformation acts nibblewise, it preserves the natural 4-bit partitioning of the state.

2.1.4 MixColumn. The linear diffusion layer processes each column of the state as a 4-dimensional vector over $\mathbb{F}_{2^4}$. For $i \in \{0,4,8,12\}$, the column

$$(s_i, s_{i+1}, s_{i+2}, s_{i+3})^T$$

is multiplied by an MDS matrix $M \in (\mathbb{F}_{2^4})^{4\times 4}$:

$$(s_i, s_{i+1}, s_{i+2}, s_{i+3})^T \leftarrow M \cdot (s_i, s_{i+1}, s_{i+2}, s_{i+3})^T.$$

The matrix M is constructed from sparse components to reduce implementation cost while providing full diffusion across the four nibbles in each column.

2.1.5 ShiftRow. The ShiftRow layer cyclically rotates each row of the state by r nibbles, where $r \in \{0,1,2,3\}$ depends on the row index. Formally,

$$\text{row } i \text{ is rotated right by } i \text{ nibble positions.}$$

Since this is a permutation on nibble indices, no mixing occurs within nibbles.

2.1.6 AddRoundKey. In each round, the 64-bit round key RK_i is XORed with the state, followed by addition of a 64-bit round constant RC_i. Round keys are generated by left-rotating K_0 or K_1 by 5 bits every two rounds. As the key schedule uses a 5-bit rotation, it does *not* respect nibble boundaries, in contrast to the data path. This forms a clear distinction relevant to our analysis.

All transformations in the round function except the key schedule operate explicitly on 4-bit nibbles. The cipher therefore exhibits a natural nibble-aligned structure in the data path, making it suitable for truncated analysis at the nibble level. This structural property is central to the method we introduce in the next section.

2.2 Definitions

Here are some useful definitions that will be used in this paper to state the notions clearly. We first provide the readers with the basic definition of an Sbox.

Definition 2.1 ($m \times n$ Sbox). An $m \times n$ Sbox is a vectorial Boolean function $S : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$ that maps an m-bit input to an n-bit output. The function S can also be expressed in terms of its component Boolean functions:

$$S(x) = (s_1(x), s_2(x), \ldots, s_n(x))$$

where each $s_i : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ is a Boolean function defining the ith output bit of the Sbox.

We denote an $n \times n$ S-box as an n-bit S-box. An n-bit S-box has n Boolean inputs and n Boolean outputs. The security of a block cipher largely depends on the algebraic properties of its nonlinear component, namely the S-box. One key property that governs resistance to differential attacks is the differential uniformity of the S-box. Below, we restate the relevant definitions following the standard literature.

Definition 2.2 (Differential Probability). Let $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be a function. For a given input difference $\delta_x \in \mathbb{F}_2^n$ and output difference $\delta_y \in \mathbb{F}_2^n$, the differential probability of the transition $\delta_x \rightarrow \delta_y$ is defined as

$$\text{DP}_F(\delta_x \rightarrow \delta_y) \;=\; \frac{\text{DDT}[\delta_x, \delta_y]}{2^n}.$$

Definition 2.3 (Truncated Differential). Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_2^n$ be sets of input and output differences, respectively. A truncated differential is a relation of the form

$$\mathcal{A} \;\Rightarrow\; \mathcal{B},$$

which holds if there exists at least one pair $(\delta_x, \delta_y) \in \mathcal{A} \times \mathcal{B}$ such that $\text{DDT}[\delta_x, \delta_y] > 0$. The probability of the truncated differential is defined as

$$\text{DP}_F(\mathcal{A} \Rightarrow \mathcal{B}) \;=\; \sum_{\delta_x \in \mathcal{A}} \sum_{\delta_y \in \mathcal{B}} \text{DP}_F(\delta_x \rightarrow \delta_y).$$

Definition 2.4 (Nibble-Based Differential Probability). Assume $n = 4m$ and let $\mathbb{F}_2^n$ be partitioned into m disjoint 4-bit nibbles. Let $\mathcal{A}_N, \mathcal{B}_N \subseteq \{0,1\}^m$ denote truncated patterns that indicate active and inactive nibbles at the input and output, respectively. The nibble-based differential probability is defined as

$$\mathrm{DP}_F^{\mathrm{nib}}(\mathcal{A}_N \Rightarrow \mathcal{B}_N) = \sum_{\substack{\delta_x \in \mathbb{F}_2^n \\ \mathrm{pat}(\delta_x) \in \mathcal{A}_N}} \sum_{\substack{\delta_y \in \mathbb{F}_2^n \\ \mathrm{pat}(\delta_y) \in \mathcal{B}_N}} \mathrm{DP}_F(\delta_x \to \delta_y),$$

where $\mathrm{pat}(\cdot)$ maps a bitwise difference to its corresponding nibble activity pattern.

2.3 MILP Formulation of XOR

Mixed Integer Linear Programming (MILP) offers a natural framework for representing combinational constraints that appear in cryptanalysis.

Definition 2.5 (Mixed Integer Linear Programming (MILP)). A Mixed Integer Linear Programming problem is defined by a matrix $A \in \mathbb{R}^{m \times n}$, a vector $b \in \mathbb{R}^m$, and a cost vector $c \in \mathbb{R}^n$. The objective is to find a vector $x \in \mathbb{Z}^k \times \mathbb{R}^{n-k} \subseteq \mathbb{R}^n$ such that

$$Ax \leq b,$$

while minimizing or maximizing the linear objective function

$$c^\top x = \sum_{i=1}^{n} c_i x_i.$$

A key element in such models is the XOR operation. Several approaches exist for encoding XOR in MILP; we begin with the straightforward, enumerative (naïve) method. Consider three binary variables $x, y, z \in \{0,1\}$ with $z = x \oplus y$. Table 1 enumerates all possible input-output combinations. Exactly half of these violate the XOR relation.

The naïve MILP encoding generates a linear inequality for each invalid tuple, ensuring that the forbidden assignments are excluded while all valid ones remain feasible.

Table 1: All tuples (x, y, z) for $z = x \oplus y$ with validity.

x	y	z	Remarks
0	0	0	valid
0	0	1	invalid
0	1	0	invalid
0	1	1	valid
1	0	0	invalid
1	0	1	valid
1	1	0	valid
1	1	1	invalid

For these four invalid assignments, the resulting MILP inequalities are:

$$\begin{aligned} x + y - z &\geq 0, \\ x - y + z &\geq 0, \\ -x + y + z &\geq 0, \\ -x - y - z &\geq -2. \end{aligned}$$

Although potential large number of inequalities, this approach provides a complete linear modelling of XOR.

2.4 XOR on n Variables via the Naïve Method

We now extend this idea to an n-input XOR. Let $(x_0, \ldots, x_{n-1}, z) \in \{0,1\}^{n+1}$ with

$$z = x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}.$$

Among the 2^{n+1} possible tuples, exactly 2^{n-1} violate this parity relation. For each invalid tuple $t = (x_0, \ldots, x_{n-1}, z)$, the naïve encoding introduces a linear constraint of the form

$$\sum_{i=0}^{n-1} (-1)^{t_i} x_i \;\geq\; \sum_{i=0}^{n-1} (-1)^{t_i} \;+\; 1, \tag{1}$$

where $t_i \in \{0,1\}$ denotes the fixed value of the variable in the invalid tuple. Constraint (1) excludes t while keeping all valid parity-consistent assignments feasible. This yields a complete MILP characterization of n-variable XOR suitable for cryptanalytic modelling.

2.5 MDS Modelling via the Even-Parity Approach

We improve upon the naïve XOR modelling by reducing the number of constraints, allowing an $(n+1)$-variable XOR to be represented using only $n+2$ variables and a single linear equation. In this context, $(a+b) \bmod 2$ is used to denote the XOR of a and b, i.e., $a \oplus b$.

Theorem 2.6 (Even-Parity Modelling [6]). *Let f be a linear Boolean function with n inputs $x_0, x_1, \ldots, x_{n-1} \in \{0,1\}$ and a single output $w \in \{0,1\}$ defined as*

$$w = f(x_0, \ldots, x_{n-1}) = \left(\sum_{i=0}^{n-1} x_i \right) \bmod 2. \tag{2}$$

Then, f can be equivalently modelled using the linear equation

$$w + \sum_{i=0}^{n-1} x_i - 2m = 0, \tag{3}$$

where $m \in \mathbb{Z}^{\geq 0}$ is an auxiliary integer variable.

Equation (3) compactly enforces the XOR constraint, as w and the x_i's sum to an even number of 1's. This formulation significantly reduces the number of inequalities compared to the naïve tuple-enumeration approach, making it especially efficient for modelling MDS layers in MILP-based cryptanalysis.

2.6 On the Use of Dummy Variables in MILP Modeling

The use of dummy variables is a common technique in MILP-based cryptanalysis and plays an important role in modeling nonlinear and linear constraints efficiently. In existing literature, three principal strategies for introducing dummy variables can be identified:

- Modeling branch number constraints in wordwise formulations, as introduced in [4];
- Representing XOR constraints of the form $x_1 \oplus \cdots \oplus x_n = 0$ using an auxiliary integer variable t such that

$$x_1 + \cdots + x_n = 2t;$$

- Introducing intermediate binary variables to decompose long XOR relations into smaller constraints, e.g.,

$$x_1 \oplus \cdots \oplus x_k \oplus a = 0, \quad a \oplus x_{k+1} \oplus \cdots \oplus x_n = 0.$$

For S-box modeling, when the algebraic structure is known and suitably compact, the third strategy is often sufficient. However, the introduction of integer dummy variables generally increases the number of integer variables in the MILP model. Since solver complexity is strongly influenced by the number of integer variables, existing works recommend minimizing their use in order to reduce running time.

In our modeling of FUTURE, we adopt the established strategy for S-box encoding from the literature. For the linear diffusion layer, however, we experimentally incorporate integer dummy variables to evaluate their practical impact on solver performance. Although theory suggests that minimizing integer variables is desirable, our experiments indicate that carefully introduced dummy integer variables can reduce the effective solving effort in certain full-round settings.

To quantify solver effort, we use the notion of *Work Units (WU)* as reported by the Gurobi optimizer. Work Units provide a hardware-independent measure of computational effort and allow consistent comparison across different modeling strategies. Our experimental evaluation demonstrates that the proposed formulation achieves improved performance in terms of WUs while maintaining exact modeling accuracy. This empirical observation suggests that, in the context of full-round MILP formulations for nibble-based SPNs, the careful use of integer dummy variables may offer a practical trade-off between model compactness and solver efficiency.

3 Nibble-based truncated modelling of FUTURE

The basic idea of this modelling approach is to treat a 4-bit unit as an abstract entity and to consider it *active* if at least one of its bits, either in the input or the output, is equal to 1; otherwise, it is considered inactive. We begin by formulating a nibble-level model that identifies an exact differential trail, thereby determining the set of potentially active S-boxes. This information is then propagated to the bitwise MILP model, where we optimize the differential probability of the trail. By restricting the search to S-boxes confirmed to be active, we avoid exploring the full combinatorial space, resulting in a substantial reduction in computational effort and a significant acceleration of the search process. The Figure 1 highlights the formulation.

3.1 S-box modelling

Let $S : \{0,1\}^4 \to \{0,1\}^4$ be a 4-bit S-box. For an input nibble $x = (x_0, x_1, x_2, x_3)$ and output nibble $y = S(x) = (y_0, y_1, y_2, y_3)$, define the activity indicator α as:

$$\alpha(x, y) = \begin{cases} 1, & \text{if } \bigvee_{i=0}^{3} x_i = 1 \text{ or } \bigvee_{j=0}^{3} y_j = 1, \\ 0, & \text{otherwise,} \end{cases}$$

where $\bigvee$ denotes the logical OR over the bits of the nibble.

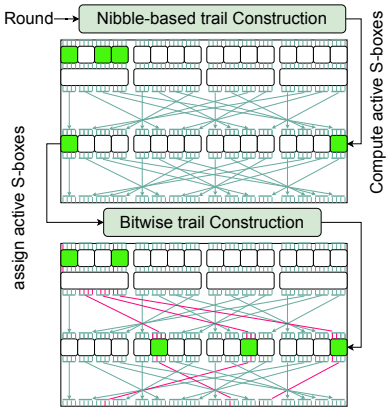

Figure 1: Trail construction technique

3.2 MixColumn Modelling

Consider the MDS matrix M that maps a 16-bit input to a 16-bit output:

$$M : \{0,1\}^{16} \to \{0,1\}^{16}.$$

We partition both the input and output into 4 nibbles (4-bit blocks):

$$x = (x_0, x_1, x_2, x_3), \quad y = M(x) = (y_0, y_1, y_2, y_3),$$

where each $x_i, y_i \in \{0,1\}^4$.

We then define the *active nibble* indicator function $\alpha : \{0,1\}^4 \to \{0,1\}$ as

$$\alpha(z) = \begin{cases} 1, & \text{if at least one bit of } z \text{ is } 1, \\ 0, & \text{otherwise.} \end{cases}$$

Using this notation, the activity of each output nibble is given by

$$\alpha(y_i) = \alpha\big(M_i(x_0, x_1, x_2, x_3)\big), \quad i = 0, 1, 2, 3,$$

where M_i denotes the mapping from the input nibbles to the i-th output nibble. A nibble is considered *active* if $\alpha(y_i) = 1$, and *inactive* otherwise.

Generate Inequalities. We generate the inequalities directly using SageMath [12]. The procedure works by representing input-output propagations as points in a $2n$-dimensional space for an n-bit mapping.

For example, if the input `0b1110` propagates to `0b0100` and `0b1000`, we represent these propagations as the points

`[1,1,1,0,0,1,0,0]` and `[1,1,1,0,1,0,0,0]`

and add them to the `Points` list. This method can be extended to any number of input/output linear or differential propagations.

```
# The Python code starts here #
# Define the points
Points = [[0, 0, 0, 0, 0, 0, 0, 0],
          ...,
          [1, 1, 1, 1, 1, 1, 1, 1]]

# Create a polyhedron using the points as vertices
triangle = Polyhedron(vertices=Points)

# Iterate through the inequality generators of the
# polyhedron and print them
for l in triangle.inequality_generator():
    print(str(l)[14:])
# The code ends here #
```

The output of this script provides all the inequalities that describe the convex hull of the specified input-output propagations. These inequalities can then be used for linear programming or other cryptanalytic purposes. For the reference, the truncated output is shown in the Appendix A.

3.3 ShiftRow Modelling

For the `ShiftRow` operation, we model it as a simple permutation of the 16 nibbles in the cipher state. Each nibble is moved to a new position according to the following mapping:

$$s_i \longleftarrow 13\, s_i \mod 16, \quad i = 0, \ldots, 15.$$

For reference, the output is provided in Appendix B.

3.4 Objective function

We use as objective function the total number of active input nibbles to the MDS layer. Let x_i denote the binary activity variable associated with the i-th input to M. For a trail spanning R rounds, the objective function is defined as

$$\min \sum_{i=0}^{16R-1} x_i.$$

4 Construction of bitwise differential trails

We model the difference distribution table (DDT) of the S-box S using a set of twelve variables. The input difference (a_0, a_1, a_2, a_3) is propagated through S to the output difference (b_0, b_1, b_2, b_3) with an associated differential probability represented by (p_0, p_1, p_2, p_3). Since each DDT entry takes a value in the range $[0, 16]$, a direct encoding requires five binary variables. However, because all differential probabilities for S are even, we employ a down-scaled representation in which each entry is divided by 2. For example, a DDT value of 16 is encoded as $(1, 0, 0, 0)$, a value of 8 as $(0, 1, 0, 0)$, a value of 6 as $(0, 0, 1, 1)$, and so forth.

4.1 Objective function

In the bitwise trail construction, we adopt an opposite objective compared to the nibble-based model. Instead of minimizing activity, we maximize the overall trail probability. Each S-box transition probability is represented using the down-scaled encoding factor of 2, using the binary variables $p_i \in \{0, 1\}$ corresponding to the bits of the 5-bit scaled DDT entry.

The objective function is therefore defined as

$$\max \sum_{i=0}^{64R-1} w_i\, p_i,$$

where the weights $w_i \in \{1, 2, 4, 8\}$ reflect the positional value of each probability bit. Explicitly, for every group of four variables $(p_{4j}, p_{4j+1}, p_{4j+2}, p_{4j+3})$ corresponding to the j-th S-box, the contribution is

$$8p_{4j} + 4p_{4j+1} + 2p_{4j+2} + p_{4j+3}, \qquad j = 0, 1, \ldots, 15.$$

This encoding ensures that maximizing the objective is equivalent to maximizing the differential probability of the full bitwise trail. A sample snippet of the objective function for round 1 is shown in Appendix C.

4.2 Valid inequalities for DDT entries

We impose all linear constraints necessary to ensure that only valid DDT transitions are permitted in the model, while all invalid transitions are eliminated. Furthermore, we incorporate the active S-box indicators $s_i \in \{0, 1\}$, obtained from our nibble-based trail construction, by assigning $s_i = 1$ for every S-box activated in the current model instance. To model the DDT of an S-box within our MILP framework, we retain only the valid differential transitions and eliminate all non-valid ones. Let $\mathcal{V} \subseteq \{0, 1\}^n$ denote the set of valid n-bit input vectors corresponding to permissible DDT entries. For every invalid vector $\mathbf{v} \in \{0, 1\}^n \setminus \mathcal{V}$, we introduce a linear constraint of the form

$$\sum_{i=0}^{n-1} (-1)^{1-\mathbf{v}_i} x_i \;\geq\; -\big(\mathrm{wt}(\mathbf{v}) - 1\big),$$

where x_i are binary MILP variables representing the input differences, $\mathbf{v}_i$ denotes the i-th bit of $\mathbf{v}$, and $\mathrm{wt}(\mathbf{v})$ is the Hamming weight of $\mathbf{v}$. This inequality prevents the invalid pattern $\mathbf{v}$ from appearing in any feasible solution, while all vectors in $\mathcal{V}$ remain admissible. By applying this construction to every invalid vector, we obtain a compact MILP formulation that enforces exactly the valid differential transitions encoded in the DDT.

4.3 Active S-boxes in differential trails

Figure 2 illustrates the active S-boxes in a full-round differential trail of the FUTURE cipher. In the figure, green boxes indicate active S-boxes, and pink arrows show the corresponding propagation trails. Figure 2(a) depicts the active S-boxes using nibble-based modelling, while Figure 2(b) shows the bitwise trail with high differential probability.

4.4 Combinatorial Optimization of Last-Round Subkey Enumeration

Consider the last round of the cipher consisting of 16 parallel 4-bit S-boxes. Let the S-box positions be indexed by

$$\mathcal{S} = \{0, 1, \ldots, 15\}.$$

The MILP solver provides a collection of differential trails. Each trail activates a subset of last-round S-boxes. Importantly, even under a minimization objective, the solver is not guaranteed to produce trails with a fixed number of active S-boxes in the last round. Due to diffusion constraints, activating one S-box may force additional S-box activations. For example, activating S-box 13 may simultaneously activate S-boxes 4 and 5 in the same round.

Attacker's Constraint. In a key-recovery setting, the attacker must enumerate the subkey bits corresponding to the active S-boxes of a selected trail. If a trail activates k last-round S-boxes, the enumeration cost is

$$2^{4k}.$$

Hence:

$$k = 1 \Rightarrow 2^4,$$
$$k = 2 \Rightarrow 2^8,$$
$$k = 3 \Rightarrow 2^{12},$$
$$k = 4 \Rightarrow 2^{16}.$$

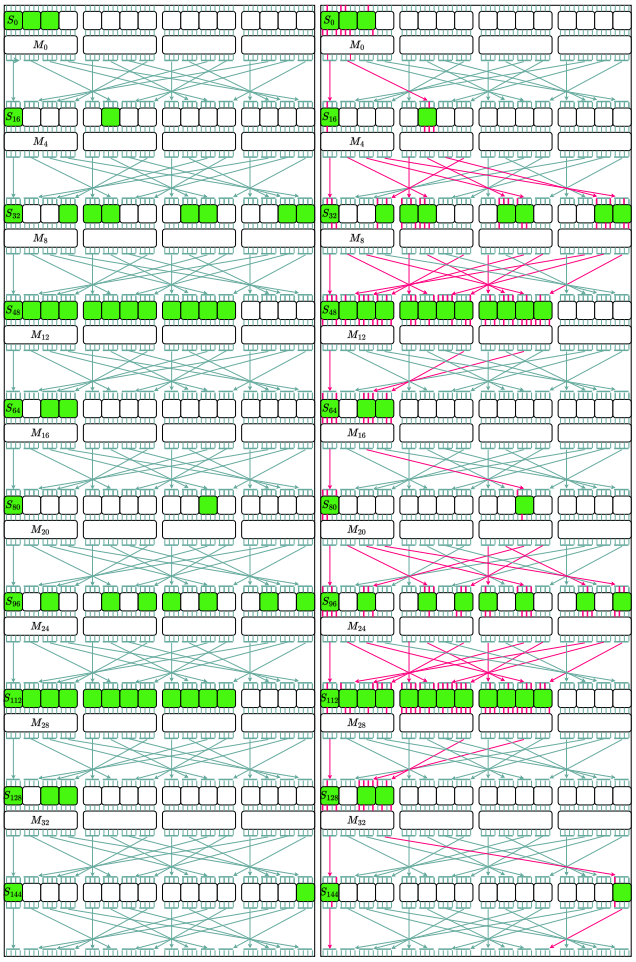

Figure 2: Active S-boxes in the differential trails.

In practice, enumeration beyond $k = 4$ becomes computationally expensive and statistically less attractive, since the signal-to-noise ratio of the differential distinguisher decreases with larger key guesses. Therefore, we restrict attention to trails with at most four active S-boxes in the last round.

Problem Formulation. Let

$$\mathcal{A}_\ell = \{A_0, A_1, \ldots, A_{m-1}\}$$

be the collection of candidate active S-box subsets obtained from the MILP solver, where

$$A_i \subseteq \mathcal{S}.$$

As a case study, suppose the solver returns

$$\begin{aligned}\mathcal{A}_\ell = \{&\{0, 15\}, \{1\}, \{2, 13\}, \{3, 14\}, \{4, 9\}, \{4, 14\},\\ &\{5, 1\}, \{6, 11\}, \{7, 8\}, \{9, 8\}, \{10, 15\},\\ &\{1, 6, 12\}, \{3, 4, 9\}, \{3, 4, 14\}\}.\end{aligned}$$

Each subset represents the active S-box positions of one candidate trail in the last round.

The attacker must select a subcollection

$$\mathcal{A}^\star \subseteq \mathcal{A}_\ell$$

such that:

(1) Every S-box position in $\mathcal{S}$ is covered at least once:

$$\bigcup_{A \in \mathcal{A}^\star} A = \mathcal{S}.$$

(2) The total subkey enumeration cost

$$\sum_{A \in \mathcal{A}^\star} 2^{4|A|}$$

is minimized.

This naturally leads to a weighted set cover optimization problem, where:

- The universe is $\mathcal{S}$,
- The candidate sets are A_i,
- The weight of a set A is $2^{4|A|}$.

Redundancy and Overlap. Redundancy arises when a small set is already contained in a larger set. For example, the singleton $\{1\}$ is redundant if $\{1, 6, 12\}$ is selected, since enumerating the latter already recovers the subkey bits corresponding to S-box 1. Thus, separately enumerating $\{1\}$ with cost 2^4 is unnecessary if $\{1, 6, 12\}$ is chosen with cost 2^{12}.

More generally, overlap between sets influences the optimal strategy.

Example A. Enumerating $\{1\}$ costs 2^4, while enumerating $\{1, 6, 12\}$ costs 2^{12}. If S-boxes 6 and 12 must also be recovered, directly choosing $\{1, 6, 12\}$ avoids three separate enumerations.

Example B. Enumerating $\{3, 14\}$ and $\{4, 9\}$ costs

$$2^8 + 2^8 = 2^9,$$

and recovers four distinct S-box positions. In contrast, enumerating $\{3, 4, 14\}$ alone costs 2^{12} and covers only three S-boxes. Hence, two smaller partially overlapping sets may be preferable to a larger one.

Example C. Enumerating $\{3, 14\}$ and $\{4, 14\}$ costs

$$2^8 + 2^8,$$

and recovers three unique S-box positions $\{3, 4, 14\}$. This may be more efficient than directly enumerating $\{3, 4, 14\}$ with cost 2^{12}.

Interpretation. The key-recovery phase therefore becomes a combinatorial optimization problem balancing:

- Coverage of all last-round S-box positions,
- Minimization of total enumeration cost,
- Control of overlap between selected trails,
- Limitation to small active-set cardinalities.

From an algorithmic perspective, the attacker seeks a minimum-weight covering family of S-box subsets under exponential weights. This formulation clarifies that optimal trail selection for key recovery is not purely a cryptanalytic question, but also a structured combinatorial optimization problem.

Trade-Off Between Trail Width and Enumeration Depth. We now address the fundamental question: is it preferable for the attacker to use many trails activating only two S-boxes, or fewer trails activating four S-boxes?

Let a trail activate k last-round S-boxes. For a 4-bit S-box, the subkey enumeration cost per trail is

$$2^{4k}.$$

Let the probability of the differential trail be p. With N chosen plaintext pairs, the expected number of right pairs is

$$\mathbb{E}[\text{right pairs}] = Np.$$

For a wrong subkey guess, the probability of satisfying the differential condition is approximately 2^{-4k}. Hence, the expected number of false positives per wrong key is

$$\mathbb{E}[\text{false pairs}] \approx N2^{-4k}.$$

To distinguish the correct subkey from wrong guesses, we require

$$Np \gg N2^{-4k},$$

which implies

$$p \gg 2^{-4k}.$$

Equivalently, the data complexity must satisfy

$$N \gtrsim \frac{1}{p}.$$

Case (A): Many Trails with $k = 2$.

Each trail requires enumeration over 2^8 subkeys. The noise level per wrong guess is approximately 2^{-8}. Thus, the distinguisher condition becomes

$$p \gg 2^{-8}.$$

Advantages:

- Small subkey enumeration cost per trail,
- Lower computational complexity,
- Faster filtering per trail.

Disadvantages:

- Each trail recovers only 8 key bits,
- More trails are required to cover all 16 S-boxes,
- Potentially larger total data complexity if individual trail probabilities are small.

Case (B): Fewer Trails with $k = 4$.

Each trail requires enumeration over 2^{16} subkeys. The noise level per wrong guess is approximately 2^{-16}. Thus, the distinguisher condition becomes

$$p \gg 2^{-16}.$$

Advantages:

- Stronger filtering capability,
- Larger statistical gap between correct and wrong keys,
- Fewer trails needed for full coverage.

Disadvantages:

- Significantly higher enumeration cost,
- Increased memory and time requirements,
- Reduced feasibility if computational resources are limited.

Data Complexity Under Chosen-Plaintext Attack. Suppose we use t trails with probabilities $p_1, \ldots, p_t$. If the trails are statistically independent, the overall success probability improves as

$$\text{Adv} \approx \sum_{i=1}^{t} Np_i,$$

while the total time complexity is

$$T = \sum_{i=1}^{t} 2^{4k_i}.$$

Hence, the attack complexity can be expressed as

$$\text{Time} \approx \sum_{i=1}^{t} 2^{4k_i}, \qquad \text{Data} \approx \max_i \frac{1}{p_i}.$$

When Does Minimization of Active S-Boxes Help? Minimizing k is beneficial when:

- Trail probabilities do not decrease significantly with smaller k,
- Computational resources are the dominant constraint,
- Multiple small trails exist with comparable probabilities.

Minimization is not beneficial when:

- Smaller trails have exponentially lower probabilities,
- Statistical signal is too weak to distinguish correct subkeys,
- The overhead of combining many trails exceeds savings in enumeration cost.

Conclusion. From an attacker's perspective, neither extreme is universally optimal. The problem reduces to balancing:

- Statistical strength (p),
- Subkey enumeration cost (2^{4k}),
- Number of trails required for full coverage,
- Available computational resources.

In practice, an optimal strategy often lies between the two extremes: selecting trails with small-to-moderate k (typically $k \leq 4$), while ensuring sufficiently high differential probabilities to maintain a clear statistical advantage.

This further reinforces that trail selection in key recovery is a joint cryptanalytic and combinatorial optimization problem.

In the following section, we formalize this selection process and integrate it into our multi-trail key-recovery framework.

5 Efficient Full Key Recovery

Classical differential key-recovery attacks typically proceed in two separate phases. First, a set of high-quality differential trails is constructed, usually by minimizing the differential weight via MILP. Second, one or more of these trails are employed in a key-recovery procedure, where the attacker enumerates subkey candidates corresponding to active S-boxes in the outer rounds.

In most existing works, these two phases are treated independently. Trails are selected solely according to their differential probability, while the cost of subkey enumeration is evaluated only afterward. However, the trail with the smallest weight is not necessarily optimal for full key recovery. In practice, the distribution of active S-boxes in the first and last rounds directly determines the number of subkey bits that must be guessed and, consequently, the overall attack complexity.

In this work, we bridge this gap by integrating trail selection and subkey-cost optimization into a unified MILP framework.

More precisely, let $\mathcal{T}(\hat{a})$ denote a collection of the best differential trails whose last-round activity is bounded by $\hat{a}$ active S-boxes. Instead of selecting a single trail, we choose a minimal subset of trails

$$\mathcal{T}^\star \subseteq \mathcal{T}(\hat{a})$$

such that:

- all required last-round S-box positions are covered,
- the induced subkey space is fully recoverable, and
- the total subkey enumeration cost is minimized.

The key idea is that different trails may activate different subsets of S-boxes in the outer rounds. By carefully combining multiple trails, it is possible to cover all necessary key-dependent components while reducing redundant guessing effort. This transforms trail selection into a combinatorial optimization problem resembling a constrained set-cover instance, where the objective function reflects the total subkey enumeration complexity rather than differential weight alone.

To formalize this strategy, we introduce binary trail-selection variables and coverage constraints within an MILP model. The objective function explicitly minimizes the number of distinct subkey bits that must be guessed, thereby directly optimizing the practical complexity of the full key-recovery attack.

The resulting formulation enables automated selection of a minimal subset of high-quality trails that collectively achieve full-round key recovery with the lowest possible subkey enumeration cost.

5.1 MILP Formulation for Multi-Trail Coverage in SPN Ciphers

Consider an SPN block cipher with block size B bits and parallel n-bit S-boxes. Let the number of S-boxes per round be

$$m = \frac{B}{n}.$$

Suppose that a collection of t high-quality differential trails has been precomputed:

$$\mathcal{T}(\hat{a}) = \{T_0, T_1, \ldots, T_{t-1}\},$$

where each trail T_i contains at most $\hat{a}$ active S-boxes in the final round.

S-box Activation Variables. For each trail T_i and each S-box position $j \in \{0, \ldots, m-1\}$ in the last round, we introduce a binary variable

$$s_j^{(i)} \in \{0, 1\},$$

where

$$s_j^{(i)} = \begin{cases} 1 & \text{if the } j\text{th S-box is active in trail } T_i, \\ 0 & \text{otherwise.} \end{cases}$$

Trail Selection Variables. We associate each trail T_i with a binary decision variable

$$\delta_i \in \{0, 1\},$$

where $\delta_i = 1$ indicates that trail T_i is selected (activated) in the key-recovery strategy.

To ensure consistency between trail selection and S-box activity, we impose

$$\sum_{j=0}^{m-1} s_j^{(i)} - |T_i| \cdot \delta_i = 0, \qquad \text{for } i = 0, \ldots, t-1,$$

where $|T_i|$ denotes the number of active S-boxes in the last round of trail T_i.

This constraint guarantees that a trail is activated if and only if all its prescribed S-boxes are active.

Coverage Variables. To model subkey recovery requirements, we introduce binary variables

$$S_j \in \{0, 1\}, \qquad j = 0, \ldots, m-1,$$

where $S_j = 1$ indicates that the jth S-box in the last round is covered by at least one selected trail.

The coverage relation is expressed via linear inequalities modeling the logical OR operation:

$$S_j \geq s_j^{(i)}, \qquad \forall i \in \{0, \ldots, t-1\},$$

$$S_j \leq \sum_{i=0}^{t-1} s_j^{(i)}, \qquad \text{for } j = 0, \ldots, m-1.$$

These constraints ensure that $S_j = 1$ if and only if at least one selected trail activates the jth S-box.

The objective function for finding the minimum number of trails is given by

$$\textbf{Minimize} \sum_{i=0}^{t-1} T_i$$

. To compute the minimum subkey enumeration cost, we consider

$$\textbf{Minimize} \sum_{i=0}^{t-1} 2^{|T_i|} \delta_i$$

.

Generality. This formulation is independent of the specific cipher structure and applies to any SPN block cipher with parallel n-bit S-boxes. It enables the joint selection of multiple differential trails while explicitly tracking S-box coverage in the final round. As shown in the next section, this mechanism forms the basis for minimizing subkey enumeration cost by selecting a subset of trails that covers all necessary key-dependent S-box positions with minimal overall complexity. An example for reference is provided in Appendix D.

5.2 Optimization Criteria and Enumeration Cost Analysis

Let $m = 16$ denote the number of 4-bit S-boxes in the last round. Each candidate differential trail T_i is represented by a binary activation vector

$$\mathbf{v}_i \in \{0, 1\}^m,$$

where the j-th component equals 1 if the j-th S-box is active in the last round under trail T_i, and 0 otherwise. By construction, each trail satisfies

$$|\mathbf{v}_i|_1 = |T_i| \leq 4.$$

Let $\delta_i \in \{0,1\}$ be the trail-selection variable. The coverage constraint ensures that every S-box position in the last round is activated by at least one selected trail:

$$\sum_{i=0}^{t-1} v_{i,j}\delta_i \geq 1, \qquad \text{for all } j \in \{0,\ldots,m-1\}.$$

Minimizing the Number of Trails. A first natural objective is

$$\min \sum_{i=0}^{t-1} \delta_i,$$

which yields the smallest subset of trails covering all last-round S-boxes. In our instance, the MILP solver returns an optimal value of 4, meaning that four trails suffice to cover all 16 S-box positions. Since each selected trail activates at most 4 S-boxes, and each S-box contributes 4 key bits, the resulting subkey enumeration complexity is bounded by

$$4 \cdot 2^{4\cdot 4} = 2^{18}.$$

Selected Trails. Let the MILP solver return the optimal trail subset

$$\mathcal{T}^\star = \{T_0, T_1, T_2, T_3\}.$$

Each trail T_i is characterized by the set of active S-box positions in the last round.

Denote by $\mathcal{A}_i \subseteq \{0,\ldots,15\}$ the active S-box indices under T_i. From the solver output, we obtain:

$$\begin{aligned}
\mathcal{A}_0 &= \{1,6,11,12\},\\
\mathcal{A}_1 &= \{0,5,10,15\},\\
\mathcal{A}_2 &= \{2,7,8,13\},\\
\mathcal{A}_3 &= \{3,4,9,14\}.
\end{aligned}$$

Observe that

$$\bigcup_{i=0}^{3} \mathcal{A}_i = \{0,\ldots,15\}, \qquad \mathcal{A}_i \cap \mathcal{A}_j = \varnothing \text{ for } i \neq j.$$

Hence, the four selected trails form a partition of the 16 last-round S-box positions. Since each S-box contributes 4 subkey bits, each trail requires enumeration over $2^{4\cdot 4} = 2^{16}$ subkey candidates. The total enumeration cost is therefore

$$4 \cdot 2^{16}.$$

These sets are used in Algorithm 1 to recover the full last-round subkey.

Minimizing Subkey Enumeration Cost. However, minimizing the number of trails does not necessarily minimize key-recovery complexity. The enumeration cost associated with a trail T_i is proportional to $2^{4|T_i|}$, since each active 4-bit S-box introduces 4 key bits. We therefore consider the refined objective

$$\min \sum_{i=0}^{t-1} 2^{|T_i|}\delta_i,$$

which directly models the total subkey enumeration effort.

Under this objective, the MILP solver produces a different optimal solution with total cost

$$256 = 16 \cdot 2^4.$$

This corresponds to selecting 16 trails, each activating exactly one S-box in the last round. Consequently, the total enumeration cost becomes

$$\sum_i 2^{|T_i|} = 16 \cdot 2^4 = 256,$$

which is strictly smaller than the complexity induced by the previous objective.

Interpretation. This comparison demonstrates that minimizing the number of trails and minimizing subkey enumeration cost are distinct optimization goals. The latter provides a more accurate complexity metric for full key recovery and justifies integrating enumeration-aware objectives directly into the MILP formulation.

6 Key recovery analysis

From the MILP-based differential search, we obtain a 10-round trail with 55 active S-boxes where there are 2 active S-boxes in Round 10.

Table 2: Differential characteristic for 10-round FUTURE with Differential probabilities

Round	Input difference	Differential probability
1	0x4c20 0000 0000 0000	2^{-6}
2	0x4000 0200 0000 0000	2^{-11}
3	0x600a ac00 0e80 0092	2^{-30}
4	0x74ea 31f1 2724 0000	2^{-60}
5	0x9072 0000 0000 0000	2^{-68}
6	0xd000 0000 0080 0000	2^{-74}
7	0xa060 0706 40b0 0206	2^{-96}
8	0x481d b9fe 2236 0000	2^{-129}
9	0x40fa 0000 0000 0000	2^{-136}
10	0x1000 0000 0000 0008	2^{-140}

Since FUTURE is a 64-bit cipher, the maximal number of chosen plaintext pairs is upper bounded by 2^{64}. Under a chosen plaintext model, about 2^{64} pairs can be generated. The expected number of right pairs is therefore

$$2^{64} \cdot 2^{-140} = 2^{-76},$$

which is much smaller than 1. Hence, the full 10-round trail cannot be observed directly. We therefore use the last-round partial decryption and guess the subkey bits corresponding to the two active S-boxes in Round 10. Each S-box is 4-bit, and thus the two active S-boxes involve 8 key bits. For a guess $k \in \{0,1\}^8$, we partially decrypt the two corresponding nibbles through the final `AddRoundKey` and `SubCell` layers and test whether the induced input difference matches the predicted difference from Round 9. The correct guess preserves the differential with probability 2^{-140}, whereas a wrong guess follows a random distribution with probability close to 2^{-8}. To recover the remaining key bits, we construct another high-probability differential trail activating the other two S-box positions in the last round and repeat the same procedure as given in Algorithm 1. This allows recovery of the remaining 8 key bits of the last round. Since the key schedule uses 5-bit rotations of K_0 and K_1, the master key can then be reconstructed uniquely by inverting the rotation relations.

Algorithm 1 Full Key Recovery on 10-Round FUTURE Using Multi-Trail Enumeration

Require: Four selected differential trails $\mathcal{T}^\star = \{T_0, T_1, T_2, T_3\}$ covering all 16 last-round S-box positions
Ensure: Recovery of the last-round subkey $K^{(10)}$

```
Generate 2^64 plaintext pairs with the required input difference.
Encrypt all pairs through 10 rounds and store the ciphertext pairs.
for each trail T_i ∈ T* do
    Let A_i ⊆ {0, ..., 15} denote the active S-box positions in Round 10 under T_i.
    Initialize a counter array indexed by subkey candidates k_i ∈ {0,1}^{4|A_i|}.
    for each subkey candidate k_i do
        Partially decrypt the ciphertext pairs through the S-boxes indexed by A_i using k_i.
        Compute the resulting difference at the input of Round 10.
        if the difference matches the predicted Round 9 difference under T_i then
            Increment the counter corresponding to k_i.
        end if
    end for
    Select the subkey candidate k_i* with the maximum counter value.
    Fix the recovered subkey bits at positions A_i.
end for
Combine all recovered subkey nibbles to obtain the full Round 10 subkey K^(10).
Invert the key schedule to recover the master key (K_0, K_1).
```

6.1 Time and Data Complexity Under Different Trail Widths

We now compare the attack complexity under two extreme design choices for the last-round differential trails:

(A) A large number of trails with $k = 1$ active 4-bit S-box,
(B) A smaller number of trails with $k = 4$ active 4-bit S-boxes.

Let k denote the number of active S-boxes in the last round for a given trail. For each trail, the attacker must enumerate 2^{4k} subkey candidates. Assume that N chosen plaintext pairs are available and that the probability of the differential trail is p.

For the correct subkey guess, the expected number of surviving pairs is

$$\mu_{\text{right}} = Np.$$

For a wrong subkey guess, the difference at the S-box input is uniformly random, and hence the probability of satisfying the predicted difference is 2^{-4k}. Thus, the expected number of false pairs per wrong key is

$$\mu_{\text{wrong}} = N2^{-4k}.$$

Successful key recovery requires a clear statistical separation:

$$Np \gg N2^{-4k} \quad \Longleftrightarrow \quad p \gg 2^{-4k}.$$

Case (A): Many Trails with One Active S-box ($k = 1$). Each trail requires enumeration over 2^4 subkey candidates. If t independent trails target distinct S-box positions, the total time complexity becomes

$$T_A = t \cdot 2^4 \cdot N.$$

Since $k = 1$, the noise level per wrong key is 2^{-4}. The statistical condition for distinguishing the correct key becomes

$$p \gg 2^{-4}.$$

If the trails are independent and their counters are accumulated, the total expected number of right pairs is

$$\mu_{\text{right}}^{(A)} = N \sum_{i=1}^{t} p_i.$$

Assuming equal probabilities $p_i = p$,

$$\mu_{\text{right}}^{(A)} = Ntp.$$

The probability that the correct key achieves the highest counter can be approximated using standard hypothesis testing. When Ntp significantly exceeds $Nt2^{-4}$, the success probability approaches 1. However, if p is close to 2^{-4}, a large number of trails or larger N is required.

Time Complexity (A)..

$$T_A = t \cdot 2^4 \cdot N.$$

Data Complexity (A)..

$$N \gtrsim \frac{1}{\sum_{i=1}^{t} p_i}.$$

If the block size is 64 bits,

$$N \leq 2^{64}$$

under a full codebook chosen-plaintext attack.

Case (B): Fewer Trails with Four Active S-boxes ($k = 4$). Each trail requires enumeration over 2^{16} subkey candidates. If t' trails are used, the time complexity becomes

$$T_B = t' \cdot 2^{16} \cdot N.$$

Here, the noise level per wrong key is 2^{-16}, which is significantly smaller than in Case (A). The statistical condition becomes

$$p \gg 2^{-16}.$$

The expected number of right pairs is

$$\mu_{\text{right}}^{(B)} = N \sum_{i=1}^{t'} p_i.$$

Since wrong keys survive with probability 2^{-16}, the gap between correct and wrong counters is much larger, leading to a higher probability of correct key ranking, even for moderate N.

Time Complexity (B)..

$$T_B = t' \cdot 2^{16} \cdot N.$$

Data Complexity (B)..

$$N \gtrsim \frac{1}{\sum_{i=1}^{t'} p_i}.$$

Table 3: Comparison of WUs for various methods.

Computing minimum active Sboxes										
Methods vs rounds	1	2	3	4	5	6	7	8	9	10
[6] with 2^{n-1} XOR eq.	0.06	5.84	1419.19	187,394.79	28,820,542.53	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
[9] with 2^{n-1} XOR eq.	0.07	2.22	157.77	6,547.64	310,847.99	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
[9] with Even-Parity XOR Modelling	0.01	0.52	35.88	2012.23	119,689.80	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
[6] with Even-Parity XOR Modelling	0.06	0.68	17.43	259.30	4,419.49	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
Computing bitwise high probability trail										
Nibble-wise trail construction	0.00	0.01	0.02	0.03	0.02	0.03	0.03	0.05	0.05	0.05
Bit-wise trail construction	0.85	10.99	96.66	12551.88	1351.92	2369.46	11810.01	22538.15	19849.59	22185.79

Comparison in Terms of Correct Key Probability. Let Δ denote the expected statistical gap:

$$\Delta = N(p - 2^{-4k}).$$

- For $k = 1$, the noise term 2^{-4} is relatively large, reducing the statistical gap.
- For $k = 4$, the noise term 2^{-16} is negligible, significantly increasing the statistical gap.

Hence:

- Case (A) is computationally cheaper but statistically weaker.
- Case (B) is computationally expensive but statistically stronger.

The expected probability of correct key recovery approaches 1 when

$$N(p - 2^{-4k}) \gg \sqrt{N2^{-4k}},$$

which follows from a normal approximation of the binomial distribution.

Conclusion. Case (A) is suitable when:

- Many moderate-probability single-S-box trails exist,
- Computational cost is the dominant constraint.

Case (B) is suitable when:

- High-probability wider trails exist,
- Strong statistical separation is required,
- Larger computational resources are available.

In practice, the optimal attack balances enumeration cost (2^{4k}) and statistical strength (p), selecting trails with small-to-moderate k that maximize the ratio

$$\frac{p}{2^{-4k}}.$$

This ratio captures the trade-off between differential strength and subkey search complexity.

7 Reduced Work Units (WUs)

In this work, we adopt *Work Units (WUs)* as a solver-level metric to quantify the computational effort required to construct differential trails within the MILP framework. Unlike time or data complexity of a cryptanalytic attack, WUs measure the internal optimization effort expended by the MILP solver during model resolution.

More precisely, WUs capture the computational burden induced by:

- the number of variables and linear constraints,
- the combinatorial structure of XOR and MDS encodings,
- and the branch-and-bound exploration required to obtain optimal solutions.

This metric enables a fair comparison between different modelling strategies independent of implementation-specific timing measurements.

Our experimental evaluation shows that:

- naïve bitwise XOR modelling incurs a rapidly increasing number of WUs as the number of rounds grows,
- Even-Parity modelling significantly reduces the number of constraints but still operates at the bit level,
- the proposed nibble-based truncated framework achieves a drastic reduction in WUs across all rounds.

In particular, while bitwise trail construction exhibits near-exponential growth in solver effort, the nibble-aligned abstraction maintains a nearly constant and negligible WU profile even for the full 10-round instance. This observation indicates that the primary scalability bottleneck lies in the modelling granularity rather than in the MILP framework itself.

Therefore, WUs serve as a principled complexity indicator demonstrating that nibble-level abstraction enables practical and scalable full-round differential analysis.

Table 3 compares the WUs required for various methods across multiple rounds. It can be observed that the methods 3 and 4 employing Even-Parity Modelling require significantly fewer WUs compared to methods 1 and 2. This reduction indicates that Even-Parity Modelling offers a more efficient formulation, leading to lower computational complexity and improved scalability as the number of rounds increases.

Remark 1. The proposed trail construction technique can be naturally extended to the construction of linear trails as well. In this setting, we consider the propagation of linear masks instead of differentials. Specifically, linear propagation through the MDS layer is captured by the relation [9]

$$\alpha M^{-1} = \beta,$$

where α and β denote the input and output linear masks, respectively. This contrasts with the differential case, where propagation through the MDS layer is expressed as

$$M \cdot \Delta x = \Delta y,$$

for input and output differences Δx and Δy.

8 Conclusion

In this work, we revisited truncated cryptanalysis from a fresh perspective by modeling SPN ciphers at the nibble level. Treating each 4-bit nibble as a single abstract unit allows us to preserve the structure and behavior of the cipher, while avoiding the heavy computational cost that comes with bit-level modeling. Using this idea, we were able to analyze FUTURE across all rounds and obtain meaningful differential and linear trails with far less effort than traditional approaches.

Our results show that nibble-based modeling can serve as a practical and efficient alternative for studying modern 4-bit SPN designs. We believe this perspective offers a helpful middle ground between fine-grained but expensive bit-level methods and coarse-grained truncated techniques, and we hope it will prove useful for future evaluations of lightweight symmetric primitives.

References

[1] Ahmadian, Z., Khalesi, A., M'foukh, D., Moghimi, H., and Naya-Plasencia, M. Truncated differential cryptanalysis: new insights and application to qarmav1-n and qarmav2-64. *Designs, Codes and Cryptography 92*, 12 (2024), 4549–4591.

[2] Aref, M. S. R. An improved truncated differential cryptanalysis of klein. *Tatra Mt. Math. Publ 67* (2016), 135–147.

[3] Avanzi, R. The qarma block cipher family. almost mds matrices over rings with zero divisors, nearly symmetric even-mansour constructions with non-involutory central rounds, and search heuristics for low-latency s-boxes. *IACR Transactions on Symmetric Cryptology* (2017), 4–44.

[4] Boura, C., and Coggia, D. Efficient milp modelings for sboxes and linear layers of spn ciphers. *IACR Transactions on Symmetric Cryptology 2020*, 3 (2020), 327–361.

[5] Crowley, P. Truncated differential cryptanalysis of five rounds of salsa20. *Cryptology ePrint Archive* (2005).

[6] Das, A. Bit-Based MILP Modelling of Non-Bit-Permutation Linear Layers for Linear Cryptanalysis. In *2024 19th Asia Joint Conference on Information Security (AsiaJCIS)* (2024), pp. 1–8.

[7] ElSheikh, M. *MILP-aided Cryptanalysis of Some Block Ciphers.* PhD thesis, Concordia University, 2021.

[8] Gupta, K. C., Pandey, S. K., and Samanta, S. FUTURE: A Lightweight Block Cipher Using an Optimal Diffusion Matrix. In *Progress in Cryptology - AFRICACRYPT 2022* (Cham, 2022), L. Batina and J. Daemen, Eds., Springer Nature Switzerland, pp. 28–52.

[9] İlter, M. B., and Selçuk, A. A. MILP-aided Cryptanalysis of the FUTURE block cipher. In *International Conference on Information Technology and Communications Security* (2022), Springer, pp. 153–167.

[10] Knudsen, L. R. Truncated and higher order differentials. In *International Workshop on Fast Software Encryption* (1994), Springer, pp. 196–211.

[11] Mouha, N., Wang, Q., Gu, D., and Preneel, B. Differential and linear cryptanalysis using mixed-integer linear programming. In *International Conference on Information Security and Cryptology* (2011), Springer, pp. 57–76.

[12] Team, T. S. D. S-Boxes and their Algebraic representations, 2024. Copyright © 2005–2024, Accessed: 2024-11-27.

[13] Yang, Q., Hu, L., Sun, S., and Song, L. Truncated differential analysis of round-reduced roadrunner block cipher. *Cryptology ePrint Archive* (2016).

A MDS layer modelling

Let the MDS matrix M take as input a 4-nibble vector $(a_0, a_1, a_2, a_3) \in \mathbb{F}_2^4$ and produce the output vector $(b_0, b_1, b_2, b_3) \in \mathbb{F}_2^4$. The set of valid transitions through M is characterized by the following system of linear inequalities:

```
+ 0 a0 + -1 a1 + 0 a2 + 0 a3 + 0 b0 + 0 b1 + 0 b2 + 0 b3 >= -1
+ -1 a0 + 0 a1 + 0 a2 + 0 a3 + 0 b0 + 0 b1 + 0 b2 + 0 b3 >= -1
+ 0 a0 + 0 a1 + -1 a2 + 0 a3 + 0 b0 + 0 b1 + 0 b2 + 0 b3 >= -1
...{trunc}...
+ 0 a12 + 1 a13 + 0 a14 + 0 a15 + 0 b12 + 0 b13 + 0 b14 + 0 b15 >= 0
+ 0 a12 + 0 a13 + 1 a14 + 0 a15 + 0 b12 + 0 b13 + 0 b14 + 0 b15 >= 0
+ 1 a12 + 1 a13 + 1 a14 + 1 a15 + 1 b12 + -4 b13 + 1 b14 + 1 b15 >= 0
```

Here a_i or b_i with value 1 is considered active nibble. The S-box corresponding to the active nibble will also be active.

B ShiftRow Modelling

We assign the values to the next consecutive row.

```
a16 - b0 = 0
a17 - b13 = 0
a18 - b10 = 0
a19 - b7 = 0
a20 - b4 = 0
a21 - b1 = 0
a22 - b14 = 0
a23 - b11 = 0
a24 - b8 = 0
a25 - b5 = 0
a26 - b2 = 0
a27 - b15 = 0
a28 - b12 = 0
a29 - b9 = 0
a30 - b6 = 0
a31 - b3 = 0
```

C Maximizing differential probability

```
Maximize
 + 8 p0 + 4 p1 + 2 p2 + 1 p3 + 8 p4 + 4 p5 + 2 p6 + 1 p7
 + 8 p8 + 4 p9 + 2 p10 + 1 p11 + 8 p12 + 4 p13 + 2 p14 + 1 p15
 + 8 p16 + 4 p17 + 2 p18 + 1 p19 + 8 p20 + 4 p21 + 2 p22 + 1 p23
 + 8 p24 + 4 p25 + 2 p26 + 1 p27 + 8 p28 + 4 p29 + 2 p30 + 1 p31
 + 8 p32 + 4 p33 + 2 p34 + 1 p35 + 8 p36 + 4 p37 + 2 p38 + 1 p39
 + 8 p40 + 4 p41 + 2 p42 + 1 p43 + 8 p44 + 4 p45 + 2 p46 + 1 p47
 + 8 p48 + 4 p49 + 2 p50 + 1 p51 + 8 p52 + 4 p53 + 2 p54 + 1 p55
 + 8 p56 + 4 p57 + 2 p58 + 1 p59 + 8 p60 + 4 p61 + 2 p62 + 1 p63
```

D Efficient subkey enumeration with the least cost

In this model, $T_i \in \{0, 1\}$ denotes whether the i-th differential trail is selected, where each trail T_i corresponds to a predefined subset of last-round S-boxes. The binary variables $s_j^{(i)} \in \{0, 1\}$ indicate whether the j-th S-box in the last round is active under the selected trail T_i. The equalities of the form $\sum_{j=0}^{3} s_j^{(i)} - 2T_i = 0$ enforce that trail T_i is activated if and only if its corresponding S-boxes are simultaneously active. The auxiliary binary variables

$$S_j = \mathrm{OR}(s_j^{(i)} | i \in [0, 3])$$

represent coverage indicators and are modelled using linear inequalities to implement logical OR relations over specific subsets of S-box variables. The constraint $\sum_j S_j = 4$ guarantees that all required S-box groups are covered by at least one selected trail. The objective function $\min \sum_i T_i$ minimizes the number of activated trails while ensuring full S-box coverage, thereby reducing the overall subkey enumeration cost in the key-recovery phase.

```
Minimize

T0 + T1 + T2 + T3

subject to

s0 + s3 - 2 T0 = 0
s5 + s7 - 2 T1 = 0
s10 + s11 - 2 T2 = 0
s13 + s14 - 2 T3 = 0

S0 - s0 >= 0
S0 - s0 <= 0

S1 - s5 >= 0
S1 - s13 >= 0
S1 - s5 - s13 <= 0

S2 - s10 >= 0
S2 - s14 >= 0
S2 - s10 - s14 <= 0

S3 - s3 >= 0
S3 - s7 >= 0
```

```
S3 - s11 >= 0
S3 - s3 - s7 - s11 <= 0

S0 + S1 + S2 + S3 = 4

Binary
```

```
T0 T1 T2 T3
s0 s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15
S0 S1 S2 S3
End
```